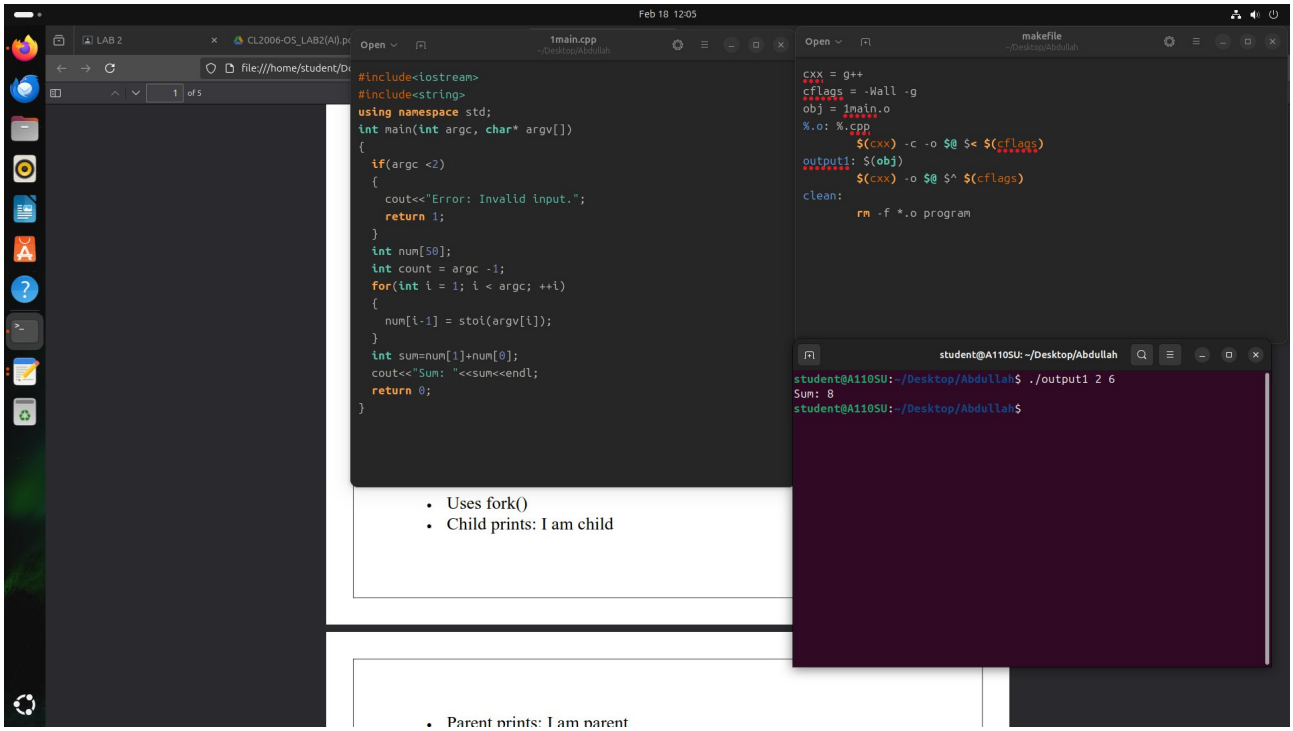
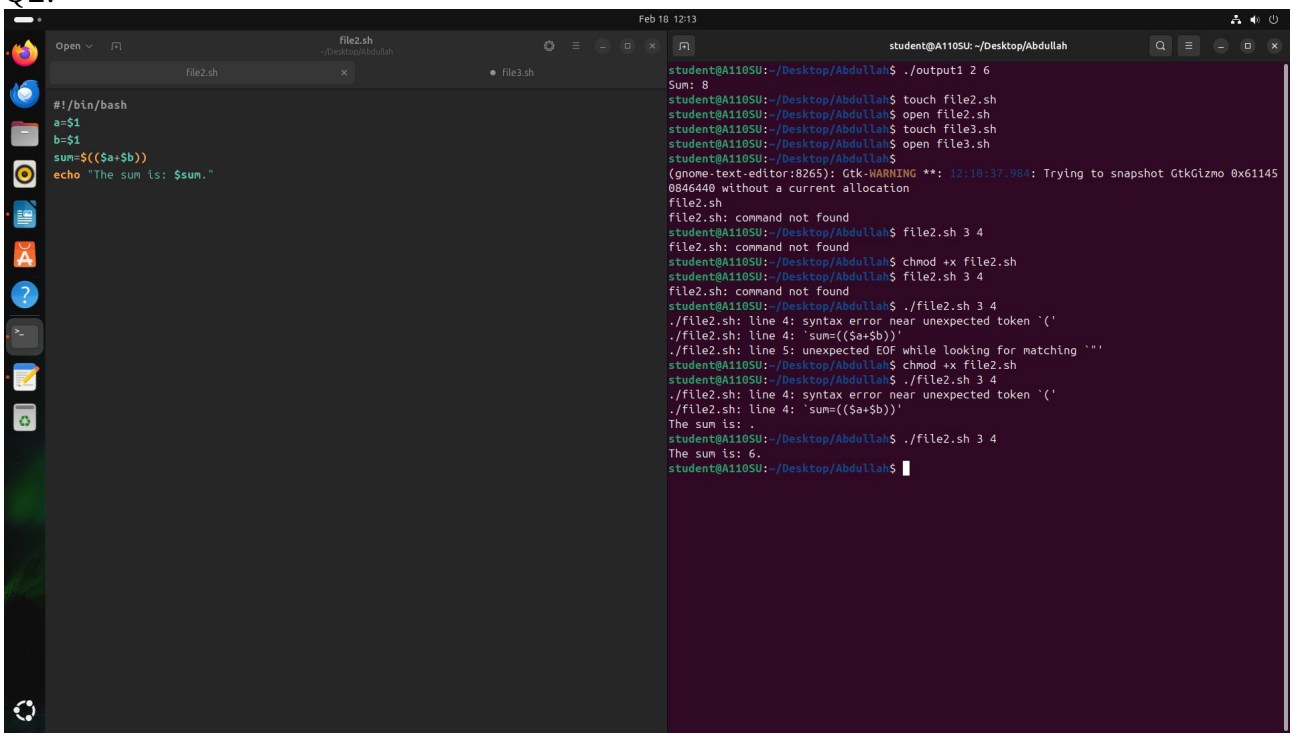
OS LAB 5 {Abdullah Sonija}

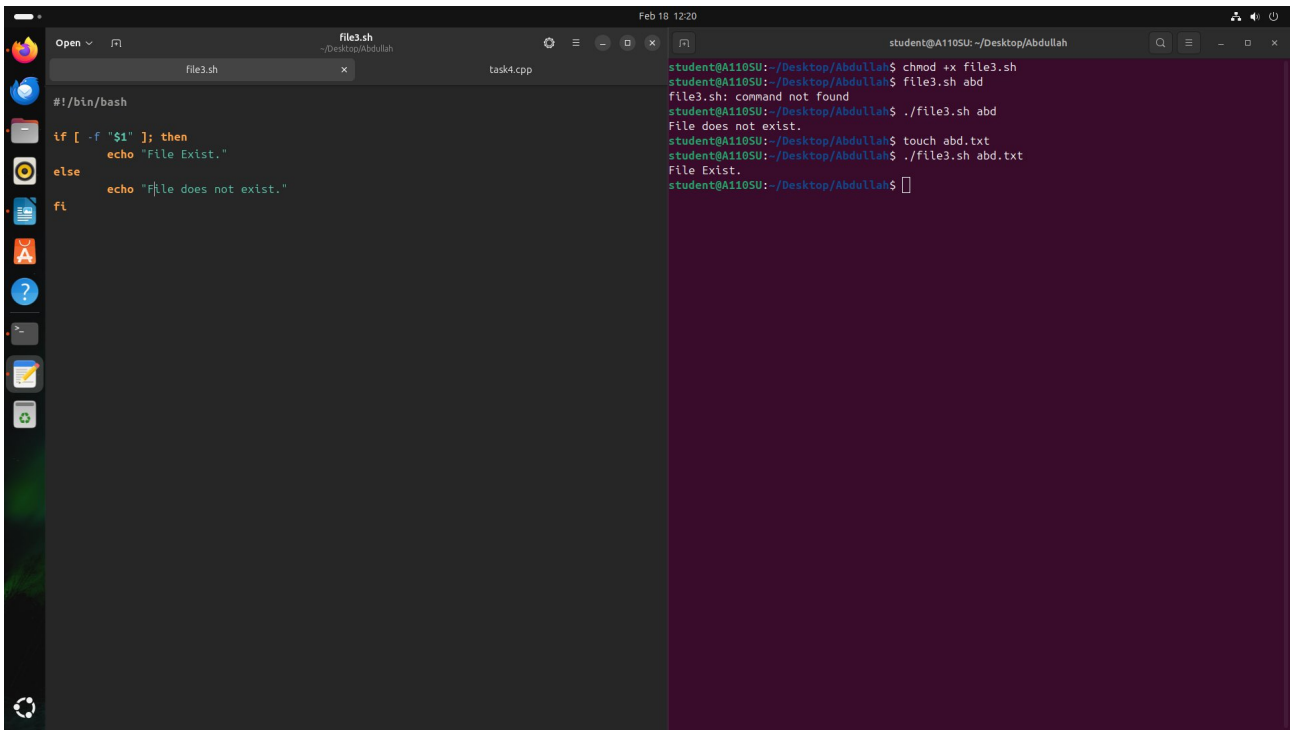
Q1:



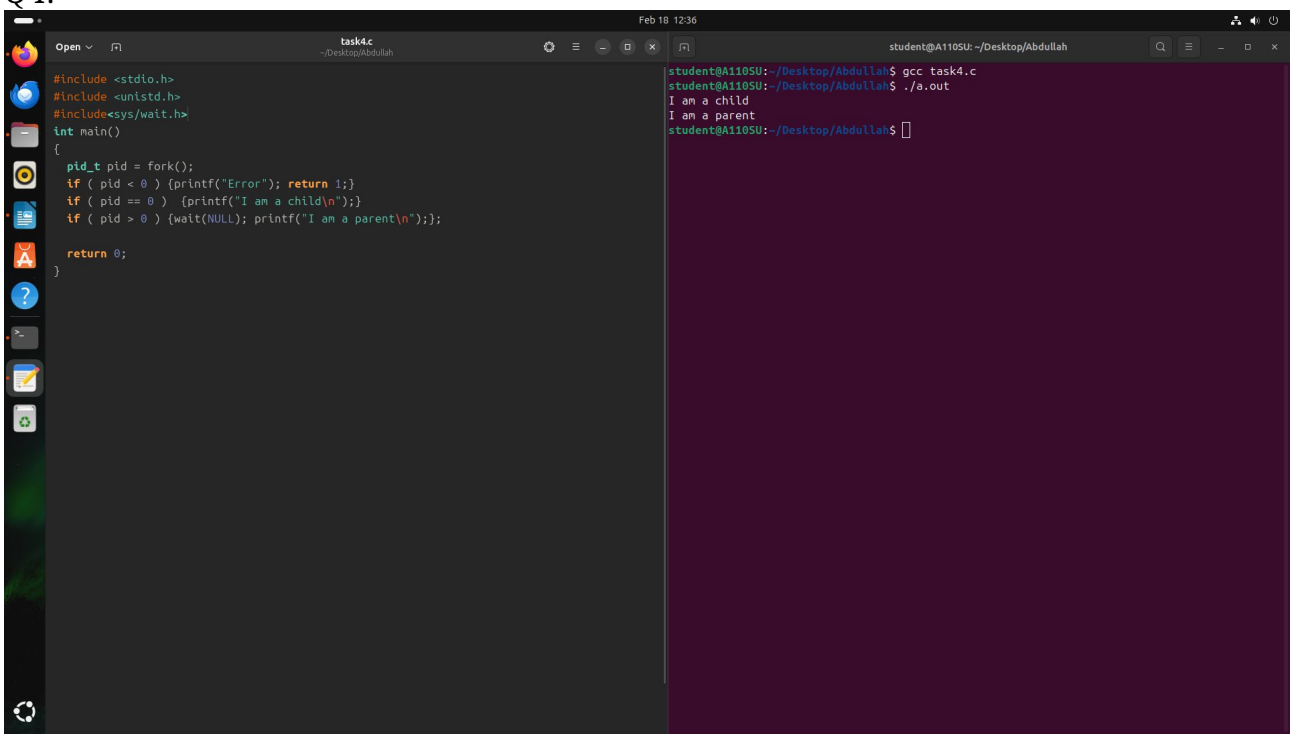
Q2:



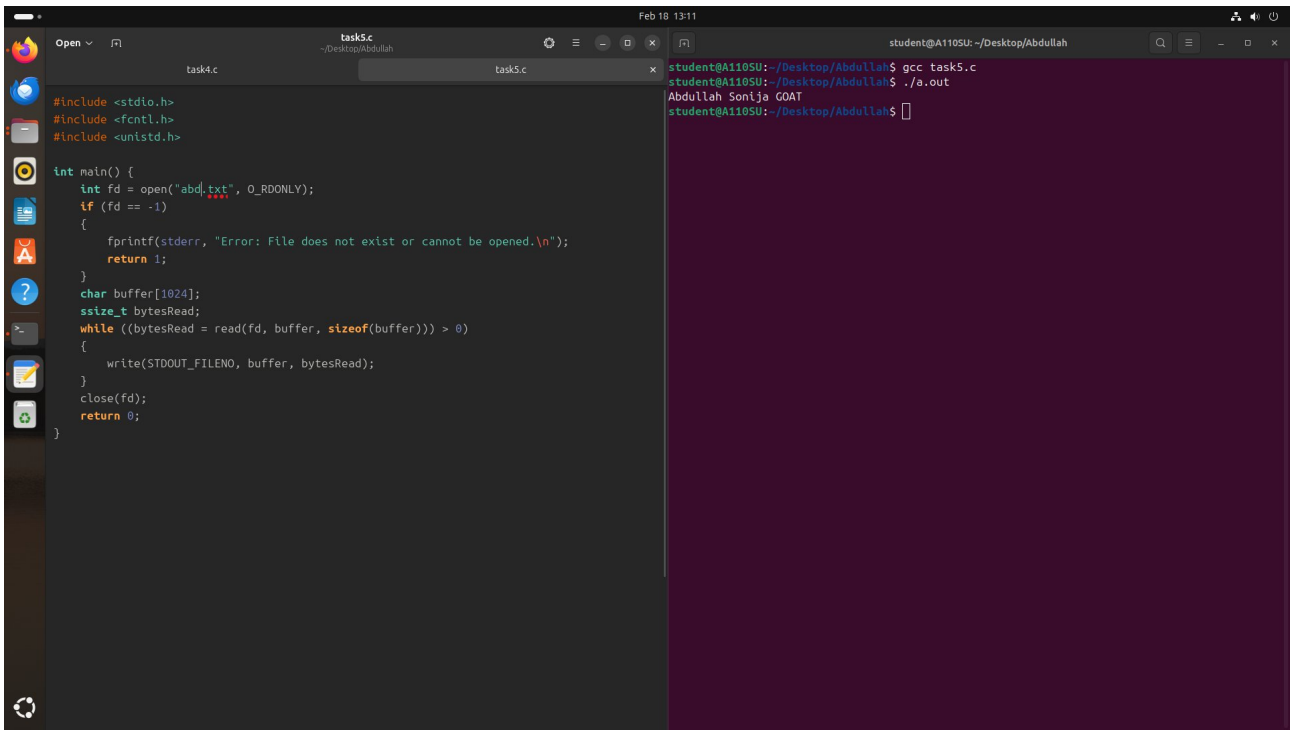
Q3:


Feb 18 12:20
Open
file3.sh
~/Desktop/Abdullah
file3.sh
task4.cpp
#!/bin/bash
if [-f "$1"]; then
echo "File Exist."
else
echo "File does not exist."
fi
student@A110SU: ~/Desktop/Abdullah
student@A110SU:~/Desktop/Abdullah$ chmod +x file3.sh
student@A110SU:~/Desktop/Abdullah$ file3.sh abd
file3.sh: command not found
student@A110SU:~/Desktop/Abdullah$ ./file3.sh abd
File does not exist.
student@A110SU:~/Desktop/Abdullah$ touch abd.txt
student@A110SU:~/Desktop/Abdullah$ ./file3.sh abd.txt
File Exist.
student@A110SU:~/Desktop/Abdullah$

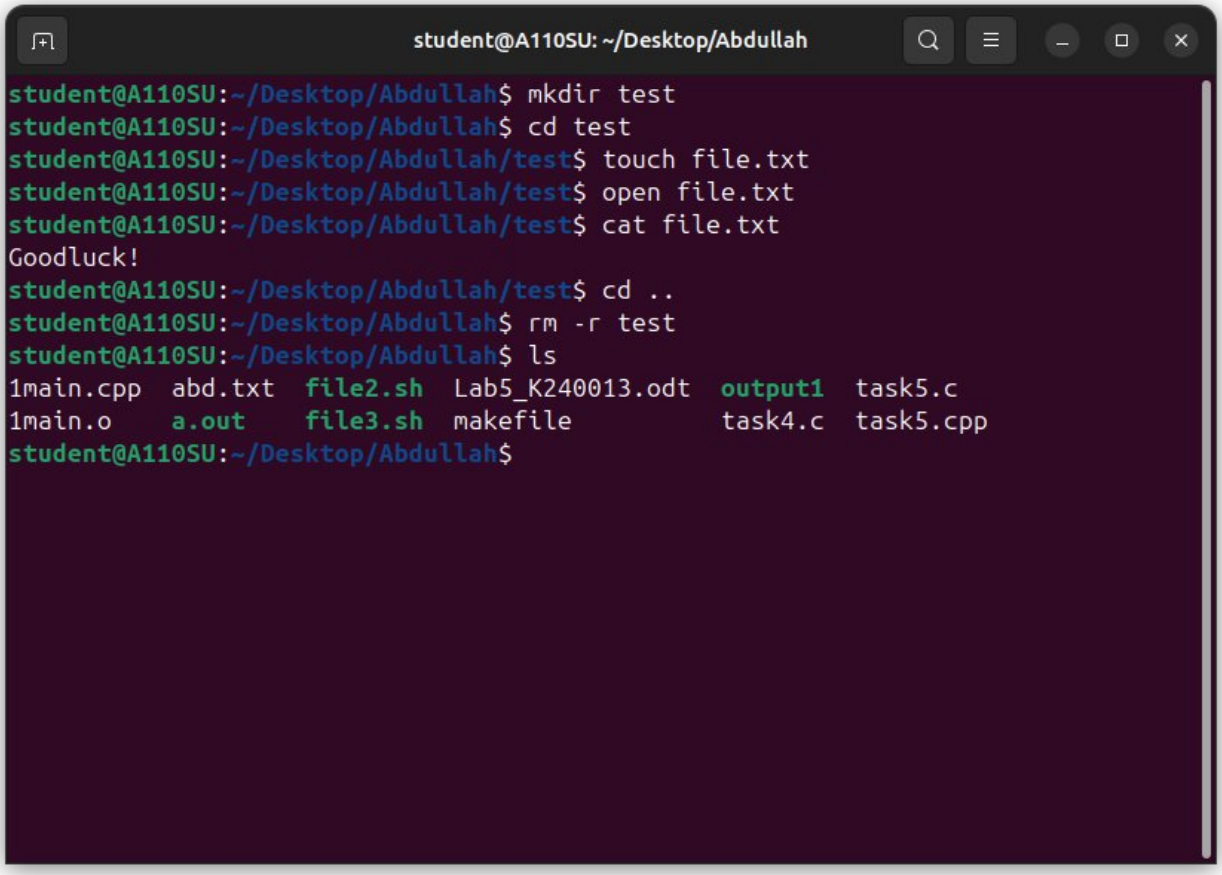
Q4:


Feb 18 12:36
Open
task4.c
~/Desktop/Abdullah
#include <stdio.h>
#include <unistd.h>
#include<sys/wait.h>
int main()
{
pid_t pid = fork();
if (pid < 0) {printf("Error"); return 1;}
if (pid == 0) {printf("I am a child\n");}
if (pid > 0) {wait(NULL); printf("I am a parent\n");};
return 0;
}
student@A110SU: ~/Desktop/Abdullah
student@A110SU:~/Desktop/Abdullah$ gcc task4.c
student@A110SU:~/Desktop/Abdullah$ ./a.out
I am a child
I am a parent
student@A110SU:~/Desktop/Abdullah$

Q5:


Feb 18 13:11
Open
task5.c
~/Desktop/Abdullah
task4.c
task5.c
student@A110SU: ~/Desktop/Abdullah
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
int fd = open("abd.txt", O_RDONLY);
if (fd == -1)
{
fprintf(stderr, "Error: File does not exist or cannot be opened.\n");
return 1;
}
char buffer[1024];
ssize_t bytesRead;
while ((bytesRead = read(fd, buffer, sizeof(buffer))) > 0)
{
write(STDOUT_FILENO, buffer, bytesRead);
}
close(fd);
return 0;
}
student@A110SU:~/Desktop/Abdullah$ gcc task5.c
student@A110SU:~/Desktop/Abdullah$ ./a.out
Abdullah Sonija GOAT
student@A110SU:~/Desktop/Abdullah$

Q6:


student@A110SU: ~/Desktop/Abdullah
student@A110SU:~/Desktop/Abdullah$ mkdir test
student@A110SU:~/Desktop/Abdullah$ cd test
student@A110SU:~/Desktop/Abdullah/test$ touch file.txt
student@A110SU:~/Desktop/Abdullah/test$ open file.txt
student@A110SU:~/Desktop/Abdullah/test$ cat file.txt
Goodluck!
student@A110SU:~/Desktop/Abdullah/test$ cd ..
student@A110SU:~/Desktop/Abdullah$ rm -r test
student@A110SU:~/Desktop/Abdullah$ ls
1main.cpp abd.txt file2.sh Lab5_K240013.odt output1 task5.c
1main.o a.out file3.sh makefile task4.c task5.cpp
student@A110SU:~/Desktop/Abdullah$

Q7:

1. What is a system call?

A system call is a programmatic way for an application to request a service from the operating system's kernel. It acts as an essential interface between a user-space process and the hardware, allowing tasks like file I/O, process creation, and network communication.

2. What is fork()?

fork() is a system call used to create a new process by duplicating the existing "parent" process. The new process, called the "child," is an exact copy of the parent (including variables and file descriptors) but has its own unique Process ID (PID).

3. What is a Makefile used for?

A Makefile is a script used by the make utility to automate the compilation and building of programs. It tracks which parts of a project have changed and only recompiles those specific files, saving significant time in large software projects.

Q8:

1. How many processes are created?
Just 2 processes. The original parent process starts first, and the fork() call creates exactly one child process.
2. How many times will "Start" print?
Only 1 time. "Start" is printed by the parent process before the fork() command is reached.
3. How many times will "OS" print?
Exactly 2 times. After fork(), both the parent and the child continue execution from that point, so both will execute the printf("OS\n") statement.

Q9:

1. Total processes created?
4 processes. Each fork() doubles the number of existing processes (2)^2 = 4
2. How many times will "Hi" print?
4 times. Since there are 4 total processes at the end, each one prints "Hi" once.
3. If one more fork() is added, how many processes?
8 processes. Adding a third fork() follows the 2^{n} rule.

Q10:

*1. What will parent print?*Parent: 3. The parent goes to the else block where x = 5 - 2.
*2. What will child print?*Child: 10. The child goes to the if block (where fork() returns 0) and calculates x = 5 + 5.

3. Do parent and child share same variable memory? Why?
No. After a fork(), the child receives a separate copy of the parent's address space. Changes made to variables in one process do not affect the other.

Q11:

1. Which prints first?"Child running". Because the parent process executes wait(NULL), it must stop and wait for the child to finish before it can move on to print "Parent done".
2. What does wait(NULL) do?
It blocks the calling process (the parent) until one of its child processes terminates.
*3. What happens if wait() is removed?*The output order becomes unpredictable (non-deterministic). The CPU might schedule the parent to finish "Parent done" before the child even prints "Child running".